

A presentation for the paper:

*“Bucket-Constrained LCS Algorithm from 2D-LIS using
Patience Sorting and Dynamic Programming”*

Awwal Mohammed*

awwalm@tuta.io

Caroline S. Selvarajah

caroline@umwales.edu.my

ACKNOWLEDGMENTS

Co-author C.S. Selvarajah, Sr. Faculty Member/Program Manager at UM-Wales, for funding this research.

IEEE/ISCI reviewers for crucial insights into the algorithms presented.



2026 IEEE Symposium on
Computers & Informatics
Concorde Hotel Kuala Lumpur, Malaysia (Hybrid)
Aug 8, 2026 to Aug 9, 2026



Pioneers in LCS Optimization



Raffaele Giancarlo
University of Palermo

Sparse DP for LCS:
Linear Cost Functions
& Convex/Concave
Functions*

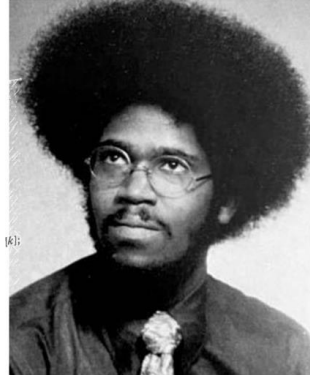
BATS Library
Implementation



Brenda S. Baker
Bell Labs

LCS from Fragments*
Clone Detection/Code
Duplication*

Parameterized Suffix
Trees



James W. Hunt
Bell Labs

Hunt-Szymanski
LCS Algorithm*

Hunt-McIlroy
UNIX **diff** File
Comparison*



Thomas Szymanski
Bell Labs

Hunt-Szymanski
LCS Algorithm*

ASSU Algorithm
for Phylogenetic
Trees



Alberto Apostolico
Georgia Tech

Improved Hunt-
Szymanski with
Sparse DP*

Suffix Tress and
Text Indexing for
LCS*

*LCS-related innovation

PRESENTATION STRUCTURE

- Background & Motivation
- Definitions & Preliminaries
- The LSBS Problem and its connection to LIS & LCS
- Our Proposed Framework
- Complexity Analysis
- Structural Parallels with Hunt-Szymanski
- Empirical Analysis & Correctness
- Summary & Conclusion

Towards a Unique LCS Framework

The LCS Problem is Everywhere

- ❖ Bioinformatics: DNA/protein alignment
- ❖ Version control: diff, merge, patch in Git, SVN
- ❖ NLP: translation alignment, plagiarism detection
- ❖ Computational biology: genomic rearrangement

THE SCALABILITY WALL

Classical DP is $\mathcal{O}(nm)$ time and space. For genomic sequences of millions of characters, this is billions of operations!

n & m – input sizes
 M – Matching list of input characters

The Gap in Existing Methods

- ❖ Hunt-Szymanski (1977): $\mathcal{O}(M \log M)$, hard to understand from first principles
- ❖ Hirschberg (1975): Linear space but still $\mathcal{O}(nm)$ time
- ❖ Baker & Giancarlo (1998-2002): pre-existing matching fragments in $\mathcal{O}(M \log \min(M, \frac{n \cdot m}{M}))$
- ❖ GPU approaches: Hardware-specific, not general
- ❖ Missing link: No clean framework connecting LCS to the well-understood 1D-LIS literature

OUR PROPOSAL

Reduce LCS to 2D-LIS . Standard **patience sorting** then yields $\mathcal{O}(M \log M)$ with full mathematical transparency.

LCS, LIS, Point Dominance

Longest Common Subsequence

DEFINITION

Given $A = (a_1, \dots, a_n)$ and $B = (b_1, \dots, b_m)$, find a maximum-length subsequence common to both.

A = "A B C B"
 B = "B C B"
 LCS = "BCB" (length 3)

Longest Increasing Subsequence

DEFINITION

Given $X = (x_1, \dots, x_n)$, find the longest strictly increasing subsequence. Solvable in $\mathcal{O}(n \log n)$ via patience sorting.

X = [3, 1, 4, 1, 5, 2, 6]
 LIS = [1, 4, 5, 6] or [1, 2, 6]

Point Dominance (Equation 1)

DEFINITION

Point $p = (p_1, \dots, p_d)$ dominates $q = (q_1, \dots, q_d)$ iff $p_i \geq q_i$ for all $i = 1, \dots, d$.

2D example:

(3, 4) dominates (1, 2): $3 \geq 1 \wedge 4 \geq 2 \checkmark$
 (2, 5) $\not\geq$ (3, 1): $2 < 3$

Each LCS of A and B corresponds to a dominance chain on match pairs (position-in-B, position-in-A). A 2D-LIS on these pairs gives the LCS. LSBS makes this explicit.

Match pair (v, i) :

v = character position in B
 i = bucket index (position in A)

Valid LCS step $\Leftrightarrow (v_j, i_j)$ strictly dominated by (v_k, i_k) : $v_j < v_k \wedge i_j < i_k$

The Longest Sorted Bucket Sequence Problem

Section IV-A (Equations 1–2)

DEFINITION

Let $\mathbf{B} = (\mathbf{B}_0, \mathbf{B}_1, \dots, \mathbf{B}_{\{n-1\}})$ be an ordered collection of buckets, each containing distinctly sorted numbers. An LSBS is a sequence $S = (s_0, s_1, \dots, s_{\{k-1\}})$ of maximum length such that for each $s_j \in S$, a pair (s_j, idx) exists where $s_j \in B_{\{idx\}}$, and:

Bucket Order: $idx_0 < idx_1 < \dots < idx_{\{k-1\}}$

Value Order: $s_0 < s_1 < \dots < s_{\{k-1\}}$

But how can it be efficiently extracted?

INTUITION

Pick at most one number from each bucket such that both indices and values strictly increase. This is a **2D-LIS on (value, bucket-index) pairs**.

Reduction from LCS to LSBS (Section VI-B)

- ❖ For each position j in A , create bucket B_j with all positions of $A[j]$ in B
- ❖ An LSBS of this CET = a common subsequence of A and B
- ❖ Longest LSBS = LCS

$A = \text{"ABCB"}, B = \text{"BCB"}$

$B_1 = \{\}, B_2 = \{1, 3\}, B_3 = \{2\}, B_4 = \{1, 3\}$

LSBS: $s_0 = 1(B_2), s_1 = 2(B_3), s_2 = 3(B_4)$

$\rightarrow \text{"BCB"} \checkmark$ (length 3 = LCS length)

Building the Common Element Table (CET)

Algorithm 1 (IV-B)

Inputs: Sequences A and B

Output: Character position buckets of A in B

```

1: let  $H$  be a new HASH TABLE
2:  $n = |A|$ ;  $m = |B|$ 
   ▼ Build character-to-indices mapping from  $B$ 
3: for  $k = 1$  to  $m$  do
4:    $char = B[k]$ 
5:   if  $char \notin H$ 
6:      $H[char] = \text{new } \textit{QUEUE}$ 
7:    $H[char] :: \text{INSERT}(k)$ 
   end for

8: let  $buckets[1..n]$  be a new ARRAY
   ▼ Build result for each character in  $A$ 
9: for  $j = 1$  to  $n$  do
10:   $char = A[j]$ 
11:  if  $char \in H$ 
12:     $buckets[j] = H[char]$ 
13:  else
14:     $buckets[j] = \emptyset$ 
15:  end for
16: return  $buckets$ 

```

Basic Example

	0	1	2	3	4	5	6	7	8	9	10	11
A:	G	T	T	C	C	T	A	A	T	A		
B:	C	G	A	T	A	A	T	T	G	A	G	A

LSBS INSTANCE

Algorithm 1 yields the following CET for A & B as a valid LSBS instance in linear time:

A	0	1	2	3	4	5	6	7	8	9
B	1	3	3	0	0	3	2	2	3	2
	8	6	6			6	4	4	6	4
	10	7	7			7	5	5	7	5
							9	9		9
							11	11		11

The Quest for a Valid LIS

Dynamic Programming (Algorithm 2)

Cross-examines each pair in the CET to determine if 2D point dominance relationship can be established.

Leverages Dynamic Programming in availing redundant re-computations.

Results in quadratic time.

OPTIMAL SUB-STRUCTURE

The problem space requires solving the same set of problems recursively, but computed optimally.

OVERLAPPING SUB-PROBLEMS

Each entry in the problem space possesses a hierarchical chain of dependence, iteratively and optimally compounding the final (and correct) solution.

Patience Sorting (Algorithm 3)

Leverages Patience Sorting and binary search over the bucket indices.

- ❖ Preds: LIS predecessor indices $\exists p_k \in \{pairs \cap LIS\}$
- ❖ Tails: Bucket index $\exists p_k \in LIS$ of *pairs*
- ❖ Positions: Index of *pairs* $\exists p_k \in LIS$ of *pairs*

CRITICAL DETAIL

Requires a bilateral sort employing a primary-ascending, and a secondary-descending criteria.

This tie-breaking prevents bucket reuse during the LIS search.

Our Unique Algorithmic Breakthrough: Bilateral Sorting Bridge to 2D-LIS

Why sort with dual criteria in complementing orders?

FUNCTIONAL TRANSITION TO LIS

- ❖ Recall that flattened pairs from the CET are grouped as (*value*, *bucket-index*)
- ❖ ALL values are PRIMARILY sorted in ascending order
- ❖ Each SUB-GROUP of bucket indices are SECONDARILY sorted in descending order
- ❖ Then a strict LIS search focusing ONLY on the bucket indices yields a 2D-LIS or more so, an LSBS!

VIDEOGAME LEADERBOARD ANALOGY

- ❖ Assume $data = [(1, 100), (2, 500), (1, 300), (2, 100)]$
- ❖ The bilaterally sorted transformation becomes:

1st $\rightarrow (1, 300)$ (Lowest level, but highest score in that level)

2nd $\rightarrow (1, 100)$

3rd $\rightarrow (2, 500)$

4th $\rightarrow (2, 100)$

Full LSBS Assembly Pipeline for LCS

Algorithm 4 (Section V-C)

Inputs: *buckets* – 2nd of CET from Algorithm 1
f – The LIS callback (Algorithm 2 or 3)
Output: The LCS of *A* and *B* as indices into *B*.

```

1: n = |buckets|
2: let flattened[1..n] be a new ARRAY

3: for i = 1 to n do
4:   for each v ∈ buckets[i] do
5:     flattened :: INSERT( (v, i) )
   end for
6: end for

7: ▼ Call Algorithm 2 or 3
8: lis = f(flattened)
9: let lcs[1..|lis|] be a new ARRAY
10: for i = 1 to |lis| do
11:   lcs[i] = B[lis[i]]
12: end for
13: return lcs

```

End-to-End Complexity

- 1** **CET Construction + Flattening**
 Hash *B*, iterate *A*, 2D grid → 1D pairs $\mathcal{O}(A + B)$
- 2a** **2D-LIS via Algorithm 2**
 Quadratic DP $\mathcal{O}(M^2)$
- 2b-I** **Bilateral Sort for Algorithm 3**
 Sort pairs: *v* ASC | *i* DESC $\mathcal{O}(M \log M)$
- 2b-II** **2D-LIS via Algorithm 3**
 Patience Sorting & Binary Search $\mathcal{O}(M \log M)$
- 3** **Backtrack preds chain**
 Recover sequence $\mathcal{O}(\max\{A, B\})$

EXPECTED PERFORMANCE

Using Algorithm 3 (2b I-II): total $\mathcal{O}(|A| + |B| + M \log M)$;
 same asymptotic class as Hunt-Szymanski, using only
 standard data structures.

Algorithm Demo

- ❖ LSBS – Dynamic Programming
- ❖ LSBS – Patience Sorting
- ❖ Wagner-Fischer
- ❖ Hunt-Szymanski

Algorithmic Complexity

ALGORITHM	TIME	SPACE	SPARSE?	KEY CHARACTERISTIC
Wagner-Fischer DP also Needleman-Wunsch 1970	$O(nm)$	$O(nm)$	No	Fills full grid; universal baseline
Hirschberg 1975	$O(nm)$	$O(n+m)$	No	Linear space via divide-and-conquer
Hunt-Szymanski 1977	$O(M \log M)$	$O(M+n+m)$	Yes	THRESH+LINK; optimal for sparse M
LSBS + Algorithm 2 1st Proposed Framework 2026	$O(M^2)$	$O(M+CET)$	Partial	Simple 2D-DP; CET-modular; clear proof; applicable to Time Series Analysis
LSBS + Algorithm 3 2nd Proposed Framework 2026	$O(M \log M)$	$O(M+CET)$	Yes	Patience sort; matches H-S; standard libs only; applicable to Time Series Analysis

Structural Parallels: LSBS ↔ Hunt-Szymanski

FUNCTIONAL EQUIVALENCE
LSBS (with Patience Sorting) and Hunt-Szymanski are **functionally equivalent** but derived from different starting points. Every LSBS component has a direct H-S counterpart.

LSBS
CET (Hash Table H + Buckets)

Hunt-Szymanski
Match List (row-ordered)

Both enumerate all M match pairs (i,j) . CET organizes by A-position; H-S by row.

LSBS
Bilateral Sort: v ASC | i DESC

Hunt-Szymanski
Row processing + reverse-column (j) order

Both prevent same-row elements from co-appearing in one LCS — equivalent tiebreaking strategies.

LSBS
tails[] array (Algorithm 3)

Hunt-Szymanski
THRESH[] array

Both store smallest ending coordinate of LIS of length k . Binary search on both $\rightarrow O(\log M)$ per step.

LSBS
enriched pairs[] + **preds**[]

Hunt-Szymanski
LINK data structure

Both record predecessor indices for backtracking. $\text{preds}[k] = \text{positions}[j-1] \leftrightarrow \text{LINK}[k] = (i,j,\text{ptr})$.

LSBS
Point dominance $(v_j, i_j < v_k, i_k \ \forall \ v < k)$

Hunt-Szymanski
Strict (*row,col*) increase

Same mathematical substrate: 2D strict dominance is the shared foundation of both algorithms.

Advantages of the LSBS Framework

vs Classical DP

- ❖ Sparse: processes only M matches, not the full $n \times m$ grid; dramatic speedup when $M \ll nm$
- ❖ Modular: CET phase separates string matching from combinatorial LIS search
- ❖ Pluggable: swap Algo 2 for Algo 3 based on M and density threshold

vs Hunt-Szymanski

- ❖ Transparent derivation: 2D-LIS formulation makes correctness visible
- ❖ Standard libraries only: hash table + bisect; no custom THRESH/LINK bookkeeping
- ❖ Open source: Python 3 implementations on GitHub, benchmarked
- ❖ Extensible: any $\mathcal{O}(n \log n)$ 1D-LIS solver can serve as callback f

Generality Beyond LCS

Any domain with ordered, non-reusable buckets and an increasing trend to find: time-series, scheduling, sampling intervals; maps directly to an LSBS instance.

FRAGMENTS PROBLEM (BAKER & GIANCARLO 1999)

Their $\mathcal{O}(M \log M)$ algorithm for the Fragments Problem is equivalent to LSBS Algorithm 3; our CET provides the conceptual bridge they did not make explicit.

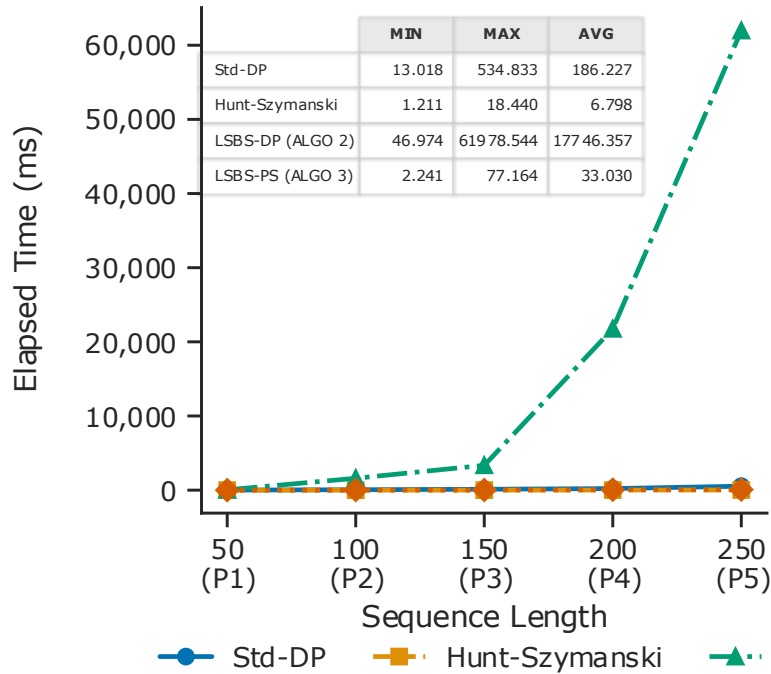
DENSE INPUT THRESHOLD

If $M \geq \lceil nm/2 \rceil$, CET space = DP table space. Although, $M \geq \lceil \frac{2}{3} nm \rceil$ is more practical. Paper recommends switching to classical DP at the latter threshold, LSBS provides the explicit criterion.

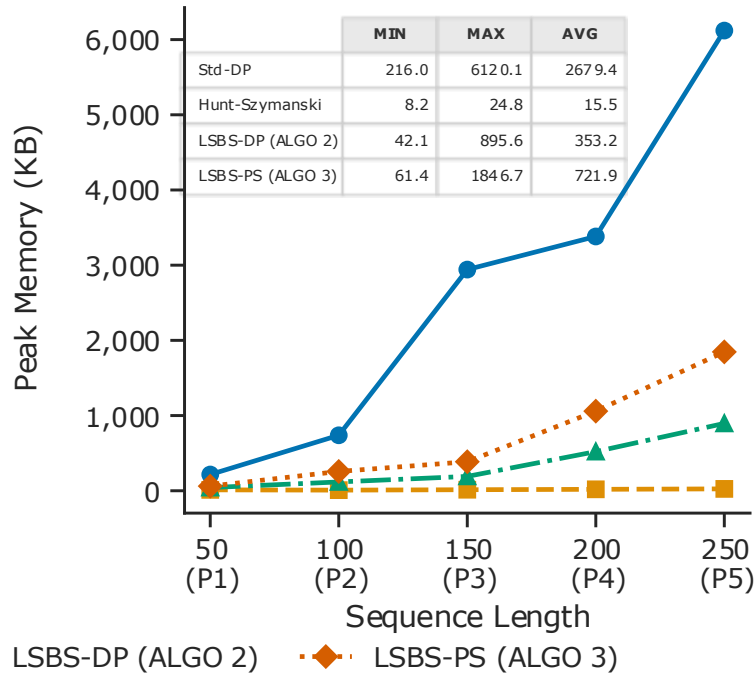
Benchmarks: Sparse ≠ GRCh38 Dataset

Five chromosome pairs at increasing lengths (50–250 chars); evenly sampled from GRCh38

LCS Benchmark – Uniform Sequence Distribution
(a) Runtime



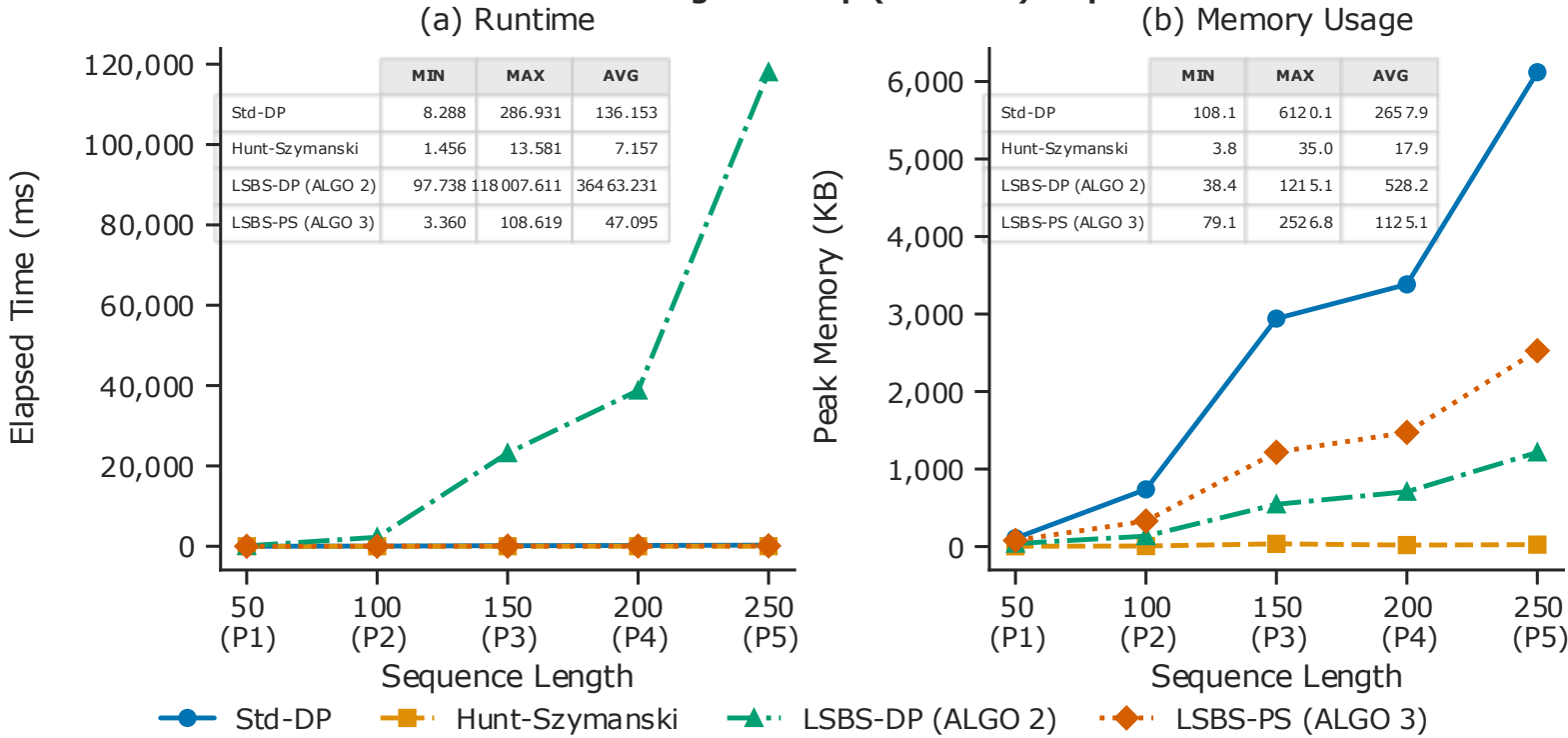
(b) Memory Usage



Benchmarks: Dense ± GRCh38 Dataset Subset

50% evenly-sampled GRCh38 sequence + 50% N-dense random fill (≈67 % N, remainder A/C/G); exposes Dense-CET degeneration

LCS Benchmark — High-Overlap (N-Dense) Sequences



What the Benchmarks Tell Us

Sparse Regime ($M \ll nm$)

LSBS VIA PATIENCE SORT EXCELS

Algorithm 3 and Hunt-Szymanski track each other closely, both $O(M \log M)$. Classical DP (SM-DP) diverges rapidly with sequence length.

- ❖ LSBS-PS: stable runtime as sequences scale past $n=200$
- ❖ CET lightweight: sparse $\rightarrow$ few matches per bucket
- ❖ Memory advantage: avoids full nm allocation

Dense Regime ($M \approx nm$)

TRADE-OFF POINT

When $M \geq \lceil \frac{2}{3} nm \rceil$, CET itself $\approx$ DP table. Algorithm 2 $O(M^2)$ becomes expensive; switch to classical DP at this threshold.

- ❖ DNA alphabet: dense CET expected; use threshold check
- ❖ Algorithm 3 still competitive even in dense cases
- ❖ Paper provides explicit density criterion in Section VI.B

Practical takeaway:

LSBS + Algorithm 3 is a drop-in replacement for Hunt-Szymanski with comparable wall-clock times and a far cleaner implementation using Python's standard `bisect` module.

Invariants & Lemma 1

Three Invariants for Algorithm 3

PREDECESSOR VALIDITY

$j = \text{BINARY-SEARCH-LEFT}(i_k)$. Since $\text{tails}[j-1] < i_k$, assigned predecessor $\text{preds}[k] = \text{positions}[j-1]$ has strictly smaller bucket-index. Formally: $i_{\text{preds}[k]} < i_k$.

OPTIMAL SUBSTRUCTURE

For every ℓ , $\text{tails}[\ell] =$ smallest bucket-index ending any increasing subsequence of length $\ell+1$. Replacement (not extension) when $j < |\text{tails}|$ preserves this (greedy optimality).

RECONSTRUCTION

$\text{positions}[L-1]$ holds index of last element of a valid LIS of length L . Backtracking via preds produces a valid LIS of length L (Lemma 1).

Lemma 1

STATEMENT

When P is bilaterally sorted as in Algorithm 3, the 2D-LIS of P w.r.t. i_k can be reconstructed by backtracking through preds starting from $\text{positions}[|\text{tails}|-1]$.

Proof sketch (three-tiered):

1. Bilateral sort: v strictly increases along any valid LIS of pairs (strict value order).
2. Same- v entries have i DESC $\rightarrow$ impossible for two to both appear in one LIS (v not strict).
3. preds chain produces (v, i) strictly increasing in both coords by construction of $\text{tails}/\text{positions}$.

Thus $\text{positions}[L-1] \rightarrow \text{preds} \rightarrow \dots \rightarrow -1$ is a valid LIS of length $L = |\text{tails}|$. ■

Summary & Contributions

MAIN CONTRIBUTIONS

- ❖ Introduced **LSBS**: a conceptual framework formally reducing LCS to 2D-LIS via the Common Element Table (CET)
- ❖ Two algorithms: $\mathcal{O}(M^2)$ **quadratic DP** (Algo 2) and $\mathcal{O}(M \log M)$ **patience sort** (Algo 3) with full correctness proofs
- ❖ Established LSBS as a new lens on **Hunt-Szymanski**: equivalent complexity, cleaner derivation, standard library implementation
- ❖ Benchmarked on GRCh38 genome dataset: LSBS-PS matches Hunt-Szymanski clock time on sparse inputs

Practical Impact

- ❖ Open GitHub: <https://github.com/awwalml/lsbs-pub> for Python implementations
- ❖ Applicable to any ordered-bucket sequential data
- ❖ Explicit density threshold criterion for algorithm selection

Future Directions

- ❖ d-dimensional extension for multi-sequence alignment (MSA)
- ❖ Parallel / GPU implementation of bilateral sort + patience phases
- ❖ Adaptive threshold & compressed CET for highly repetitive sequences

Bottom Line:

LSBS provides a principled, modular, theoretically transparent LCS framework achieving optimal $\mathcal{O}(M \log M)$ through standard algorithmic building blocks; advancing both the clarity and accessibility of LCS algorithm design.

KEY REFERENCES

- [1] Fischer & Wagner 1974: String-to-String Correction. JACM 21(1).
- [2] Erdős & Szekeres 1935: Combinatorial Problem in Geometry.
- [5] Mallows 1973: Patience Sorting. Bull. IMS 9.
- [6] Hunt & Szymanski 1977: Fast Algorithm for LCS. CACM 20(5).
- [13] Knuth 1997: TAOCP Vol. 3, §5.3.
- [14] Needleman & Wunsch 1970: Gen. Method for Proteins. JMB 48.
- [16] Hirschberg 1975: Linear Space Algorithm. CACM 18(6).
- [20] Masek & Paterson 1980: Four Russians for LCS. JCS 20.
- [22] Jaja, Mortensen & Shi 2022: Space-Efficient LIS Algos. STACS.
- [26] Baker & Giancarlo 1999: Sparse Dynamic Prog. J. Alg. 42.
- [28] Goodrich, Tamassia et al.: Data Structures & Algorithms. Wiley.
- [30] GRC 2013: Human Reference Genome GRCh38. NCBI.

Thank You

Source code and demo: **github.com/awwalM/lbsbs-pub**
Direct any questions to **awwalM@tuta.io**